

Polynomial

環狀鏈結串列

多項式報告

41243103 林采儀

2024/12/29

解題說明

本次作業要求我們實作一個 C++ 類 Polynomial，用來表示和操作具有整數係數的一元多項式。多項式使用帶有頭節點的循環鏈表來表示，每個節點包含係數、指數和指向下一個節點的指針。我們需要實現多項式的加法、減法、乘法運算，以及多項式的輸入、輸出和評估。

設計與實作細節

1. 資料結構設計

- Node 結構：
 - 每個節點儲存一項多項式：
 - coef 為係數，exp 為指數。
 - link 為指向下一個節點的指針。
 - 預設建構式初始化 coef 和 exp 為 0，並將 link 設為 nullptr。
 - Polynomial 類別：
 - 使用鏈表實作多項式的主要操作。
 - head 指向循環鏈表的頭部，避免處理空鏈表的特殊情況。
-

2. 建構與記憶體管理

- 建構式 (Polynomial())：
 - 建立 head 節點，並讓其 link 指向自身，形成空的循環鏈表。
 - 拷貝建構式 (Polynomial(const Polynomial& a))：
 - 深拷貝來源多項式 a，確保新物件與原始物件互不干涉。
 - 析構式 (~Polynomial())：
 - 循環釋放所有節點的記憶體，避免記憶體洩漏。
-

3. 多項式基本操作

- 插入項 (InsertTerm)：
 - 將指定的係數和指數插入多項式，並確保：
 1. 依指數遞減排列。
 2. 當指數相同時，合併係數。
 3. 若係數為 0，則刪除該項。
 - 邏輯：
 1. 循環遍歷鏈表，找到適當的位置。
 2. 若指數相同，直接修改係數。
 3. 若需新增節點，更新鏈表指向。

- 加法 (operator+):
 - 合併兩個多項式：
 1. 同指數相加。
 2. 高指數優先加入結果。
 3. 若某多項式遍歷完，將剩餘項目加入結果。
 - 減法 (operator-):
 - 與加法邏輯類似，但將第二個多項式的係數取反。
 - 乘法 (operator*):
 - 計算兩個多項式的乘積，對於每一對項目，將係數相乘並指數相加。
 - 運算複雜度較高，因為需要遍歷兩個鏈表。
-

4. 多項式評估

- Evaluate 函數：
 - 使用公式 $\text{result} += \text{coef} * \text{pow}(x, \text{exp})$ 計算多項式在 x 的值。
 - 遍歷每個節點，累加其對應的值。
-

5. 輸入與輸出

- 輸入 (operator>>):
 - 接收多項式的項數及每項的係數與指數，依序插入。
- 輸出 (operator<<):
 - 以數學形式輸出多項式：
 - 例如： $3x^2 + 2x^1 + 1$ 。

優點

1. 抽象化資料結構設計：

- 使用Node類別封裝每一個項，讓程式的結構更加清晰，易於維護。
- Polynomial類別提供多項式的操作，例如加法、減法、乘法和求值，使數學運算簡潔明了。

2. 彈性記憶體管理：

- 使用環狀鏈結串列來存儲多項式項目，能根據需求動態調整容量，提供靈活的記憶體管理機制。
- 在拷貝建構子和運算子重載中，使用new和delete確保正確分配與釋放記憶體，避免記憶體洩漏。

3. 多載運算子支援：

- 使用>>和<<這兩個運算子進行多項式的輸入和輸出，這樣的設計符合 C++ 語言的習慣，讓使用者能夠輕鬆操作。
-

缺點

1. 排序效率低：

- 當多項式的項目數量較多時，插入排序的時間複雜度為 $O(n^2)$ ，效率較低。

2. 合併過程性能不足：

- 目前在多項式排序後，會對每一對項目進行逐項合併，這樣的複雜度為 $O(n^2)$ 。隨著項目數量增加，合併過程的計算時間會迅速增長。

3. 動態記憶體管理成本：

- 雖然動態調整鏈表容量提高了靈活性，但在頻繁進行項目新增操作時，鏈表擴展會帶來額外的開銷。這樣的設計對記憶體管理效率要求較高，若頻繁擴展鏈表，可能會對效能產生影響。

效能分析

在本次實作中，我們使用了帶有頭節點的環狀鏈結串列來表示多項式。以下是對各個主要操作的效能分析：

1. 插入項 (InsertTerm)

- 時間複雜度： $O(n)$
 - 正確：需要遍歷鏈結串列找到正確的插入位置，最壞情況下需要遍歷所有節點，尤其當新項的指數比現有所有項的指數都小時。
 - 補充建議：如果新增的項直接插入鏈表尾端（例如插入零次方項），則時間複雜度為 $O(1)$ 。因此實際效能取決於插入位置。
 - 空間複雜度： $O(1)$
 - 正確：只需分配一個新節點的空間，與鏈表長度無關。
-

2. 加法運算 (operator+)

- 時間複雜度： $O(n + m)$
 - 正確：需要同時遍歷兩個多項式鏈結串列，並將結果存入新的鏈結串列。每個多項式的所有項都需要被處理一次。
 - 補充建議：如果多項式非常稀疏，這裡的 n 和 m 可以被視為有效的非零項數，而非總長度。
 - 空間複雜度： $O(n + m)$
 - 正確：結果多項式最多包含 $n + m$ 項，因為每一項都可能是兩個多項式中不同項的合併。
-

3. 減法運算 (operator-)

- 時間複雜度： $O(n + m)$
 - 正確：與加法運算類似，因為減法運算本質上是將第二個多項式的係數取負後再執行加法操作。
 - 空間複雜度： $O(n + m)$
 - 正確：結果多項式最多包含 $n + m$ 項，與加法運算一致。
-

4. 乘法運算 (operator*)

- 時間複雜度： $O(n * m)$
 - 正確：每一個多項式的每一項都需要與另一個多項式的每一項相乘，因此有 $n * m$ 次運算。
 - 補充建議：還需要考慮結果合併同類項的時間複雜度。如果使用鏈結串列，合併可能需要額外的遍歷操作，具體情況依實作而定。
 - 空間複雜度： $O(n * m)$
 - 正確：結果多項式最多包含 $n * m$ 個項（未合併前的所有可能乘積）。
-

5. 評估多項式 (Evaluate)

- 時間複雜度： $O(n)$
 - 正確：需要逐一遍歷多項式的 n 個項，並計算每一項的值。
- 空間複雜度： $O(1)$
 - 正確：只需儲存中間累加的結果，無需額外的空間。

測試與驗證

測試數據 & 測資設計

測資 1：基本加法與乘法測試

輸入：

- 多項式 1： $3x^2+2x+1$
- 多項式 2： x^3+x+5

預期結果：

- 加法結果： x^3+3x^2+3x+6
- 減法結果： $-x^3+3x^2+x+4$
- 乘法結果： $3x^5+2x^4+4x^3+17x^2+11x+5$

實際結果：

```
P1: 3x^2 + 2x^1 + 1x^0
P2: 1x^3 + 1x^1 + 5x^0
P1 + P2: 1x^3 + 3x^2 + 3x^1 + 6x^0
P1 - P2: - 1x^3 + 3x^2 + 1x^1 - 4x^0
P1 * P2: 3x^5 + 2x^4 + 4x^3 + 17x^2 + 11x^1 + 5x^0

P1 + P2(-1) = 5
P1 - P2(-1) = -1
P1 * P2(-1) = 6

P1 + P2(0) = 6
P1 - P2(0) = -4
P1 * P2(0) = 5

P1 + P2(1) = 13
P1 - P2(1) = -1
P1 * P2(1) = 42

P1 + P2(2) = 32
P1 - P2(2) = 2
P1 * P2(2) = 255
```

測資 2：包含零係數的多項式

輸入：

- 多項式 1： $4x^4+0x^3+2x^2$
- 多項式 2： $3x^3+0x^2+x$

預期結果：

- 加法結果： $4x^4+3x^3+2x^2+x$
- 減法結果： $4x^4-3x^3+2x^2-x$
- 乘法結果： $12x^7+10x^5+2x^3$

實際結果：

```
P1: 4x^4 + 2x^2
P2: 3x^3 + 1x^1
P1 + P2: 4x^4 + 3x^3 + 2x^2 + 1x^1
P1 - P2: 4x^4 - 3x^3 + 2x^2 - 1x^1
P1 * P2: 12x^7 + 10x^5 + 2x^3

P1 + P2(-1) = 2
P1 - P2(-1) = 10
P1 * P2(-1) = -24

P1 + P2(0) = 0
P1 - P2(0) = 0
P1 * P2(0) = 0

P1 + P2(1) = 10
P1 - P2(1) = 2
P1 * P2(1) = 24

P1 + P2(2) = 98
P1 - P2(2) = 46
P1 * P2(2) = 1872
```

測資 3：包含負係數的多項式

輸入：

- 多項式 1： $-2x^3 - 4x^2 + x - 5$
- 多項式 2： $3x^3 - x + 6$

預期結果：

- 加法結果： $x^3 - 4x^2 + 1$
- 減法結果： $-5x^3 - 4x^2 + 2x - 11$
- 乘法結果： $-6x^6 - 12x^5 + 5x^4 - 23x^3 - 25x^2 + 11x - 30$

實際結果：

```
P1:  - 2x^3 - 4x^2 + 1x^1 - 5x^0
P2:  3x^3 - 1x^1 + 6x^0
P1 + P2:  1x^3 - 4x^2 + 1x^0
P1 - P2:  - 5x^3 - 4x^2 + 2x^1 - 11x^0
P1 * P2:  - 6x^6 - 12x^5 + 5x^4 - 23x^3 - 25x^2 + 11x^1 - 30x^0

P1 + P2(-1) = -4
P1 - P2(-1) = -12
P1 * P2(-1) = -32

P1 + P2(0) = 1
P1 - P2(0) = -11
P1 * P2(0) = -30

P1 + P2(1) = -2
P1 - P2(1) = -18
P1 * P2(1) = -80

P1 + P2(2) = -7
P1 - P2(2) = -63
P1 * P2(2) = -980
```

測資 4：僅有常數項的多項式

輸入：

- 多項式 1：7
- 多項式 2：-3

預期結果：

- 加法結果：4
- 減法結果：10
- 乘法結果：-21

實際結果：

```
P1: 7x^0
P2:  - 3x^0
P1 + P2: 4x^0
P1 - P2: 10x^0
P1 * P2:  - 21x^0

P1 + P2(-1) = 4
P1 - P2(-1) = 10
P1 * P2(-1) = -21

P1 + P2(0) = 4
P1 - P2(0) = 10
P1 * P2(0) = -21

P1 + P2(1) = 4
P1 - P2(1) = 10
P1 * P2(1) = -21

P1 + P2(2) = 4
P1 - P2(2) = 10
P1 * P2(2) = -21
```

測資 5：包含相同指數的多項式項

輸入：

- 多項式 1： $5x^3 + 2x^2 + 3x^3$
- 多項式 2： $x^3 + x + 4x^3$

預期結果：

- 加法結果： $13x^3 + 2x^2 + x$
- 減法結果： $3x^3 + 2x^2 - x$
- 乘法結果： $40x^6 + 10x^5 + 8x^4 + 2x^3$

實際結果：

```
P1: 8x^3 + 2x^2
P2: 5x^3 + 1x^1
P1 + P2: 13x^3 + 2x^2 + 1x^1
P1 - P2: 3x^3 + 2x^2 - 1x^1
P1 * P2: 40x^6 + 10x^5 + 8x^4 + 2x^3

P1 + P2(-1) = -12
P1 - P2(-1) = 0
P1 * P2(-1) = 36

P1 + P2(0) = 0
P1 - P2(0) = 0
P1 * P2(0) = 0

P1 + P2(1) = 16
P1 - P2(1) = 4
P1 * P2(1) = 60

P1 + P2(2) = 114
P1 - P2(2) = 30
P1 * P2(2) = 3024
```

測試結論

1. 程式在加法與乘法操作中能正確處理零係數、負係數及相同指數項的合併。
2. 多項式的輸入輸出格式清晰，符合題目需求。
3. 排序功能正確，結果按照指數從大到小排列。
4. 求值功能準確，能正確計算指定 x 值下的多項式值。

申論及開發報告

陣列方式與鏈結方式的比較

1. 資料結構設計

陣列方式：

- 使用兩個一維陣列分別存放多項式的係數（coeff）和次數（exp）。
- 每個多項式的所有項存儲於陣列中，且次數必須按降序排列。
- 優點：結構簡單，記憶體連續分配，存取效率高。
- 缺點：需事先分配固定大小，增加或刪除項較麻煩。

鏈結方式：

- 使用節點（Node）來存儲每一項，節點包含係數、次數以及指向下一節點的指針。
- 環狀或單向鏈結可以靈活處理多項式的動態增減。
- 優點：記憶體使用靈活，適合稀疏多項式，新增或刪除項更高效。
- 缺點：指針額外消耗記憶體，節點的遍歷耗時較多。

2. 運算效率

加法運算：

- **陣列方式：**需要兩個指標分別遍歷兩個陣列，按次數進行比較與合併，時間複雜度為 $O(n+m)$ （ n, m 為兩多項式的項數）。
- **鏈結方式：**通過指針遍歷鏈結結構並進行比較與合併，時間複雜度也為 $O(n+m)$ 。但由於頻繁的動態記憶體分配，實際效率可能略低於陣列方式。

乘法運算：

- **陣列方式：**雙重迴圈遍歷兩個陣列，計算所有項的乘積，並需要在結果陣列中進行合併，時間複雜度為 $O(n \cdot m)$ 。
- **鏈結方式：**類似於陣列方式，但需頻繁進行新節點分配和動態插入操作，效率可能低於陣列方式。

3. 記憶體使用

陣列方式：

- 所需記憶體為固定大小的連續區域。
- 記憶體利用率較低，尤其對於稀疏多項式。
- 無需額外的指針存儲。

鏈結方式：

- 每個項都動態分配記憶體，按需使用。
- 每個節點需額外存儲一個指針，導致總記憶體開銷增加。
- 適合稀疏多項式，但不適合項數過多的情況。

4. 靈活性

陣列方式：

- 適合多項式項數固定、次數範圍有限的情況。
- 無法方便地處理動態新增或刪除項。

鏈結方式：

- 更靈活，支持多項式的動態增減。
 - 適用於次數範圍大、項數不確定的情況。
-

程式設計改進與探索

動態記憶體應用

本次實作中，多項式操作充分利用了動態記憶體管理。在多項式加法與乘法運算中，為每個結果項動態分配記憶體，確保運算的靈活性，同時避免了靜態記憶體分配可能帶來的浪費與限制。

運算效率優化

目前的排序演算法使用了泡沫排序（Bubble Sort），適用於小規模多項式的排序需求。然而，隨著多項式項數的增加，泡沫排序的 $O(n^2)$ 時間複雜度成為性能瓶頸。雖然程式中尚未實現改進，但理論上引入更高效的排序演算法（如快速排序）可以將效率提升至 $O(n\log n)$ ，這是一個值得探索的方向。

挑戰與解決方法

多項式項數差異處理

在多項式合併操作中，針對項數不同的情況，本次實作採用了雙指標比較法（Two-Pointer Approach）。

- 實作方法：**
同時遍歷兩個多項式，依據指數大小動態插入結果項，避免了因指數錯位而導致的合併錯誤。
 - 優點：**
這種方法簡單直觀，對於處理小規模多項式具備良好的可讀性與實用性。
-

改進方向

目前的合併操作基於雙重遍歷，時間複雜度依然較高。在未來的探索中，可以考慮引入以下改進：

- **使用哈希表**

設計一個哈希表來記錄多項式項的指數與對應係數，將兩個多項式的項插入哈希表中，最後進行合併。此方法可以將合併操作的時間複雜度降至線性 $O(n)$ 。

收穫與心得

通過本次作業，我深化了對多項式運算背後的資料結構的理解，並練習了 C++ 中動態記憶體管理和運算子重載技術的應用。

在解決具體問題時，我學會了如何權衡資料結構的靈活性與效率，並體會到設計高效程式所需的細緻與耐心。未來，我會將此次經驗應用於更多的程式開發中，持續提升自己的技術能力。

結論：

本次實作讓我理解了多項式的運算邏輯，並且通過程式優化體現了結構設計與效率平衡的重要性。我將繼續探索更高效的實作方法，為未來的學習與專案打下紮實基礎。